STUDYBOOKS · ENGINEERING GUIDE

Shipping the PWA to the iOS App Store & Google Play

Tightening the current Next.js PWA for store deployment with Capacitor: a repo-specific audit, the work required, requirements per store, and the trade-offs of each packaging route.

Project: **tt-app** (StudyBooks) · Stack: Next.js 16 · Capacitor 8 · next-auth · Stripe
Prepared: July 2026 · Bundle ID: `ee.taskutark.studybooks`

Executive summary

The project is already ~40% of the way there: **Capacitor 8 is installed and configured** (`capacitor.config.ts`, `cap:sync / cap:ios / cap:android` scripts, and `docs/MOBILE_PACKAGING.md`), a web manifest exists, and safe-area viewport handling is in place. What blocks a store submission today is not the wrapper — it is the **web app's server coupling and missing PWA fundamentals**:

- **Broken manifest:** it references `icon-192.png` / `icon-512.png`, which do not exist in `public/icons/`.
- **No service worker:** the app is not installable as a PWA and has zero offline capability.
- **Server coupling:** API routes (`/api/feed`, `/api/studybooks`, `/api/stripe/checkout`, next-auth), and server actions (onboarding, admin) are all incompatible with the static export the Capacitor shell needs.
- **Cookie-based auth:** next-auth's cookie session will not survive inside a `capacitor://` WebView origin.
- **Stripe for digital content:** the single biggest *policy* risk — Apple requires In-App Purchase for digital goods in most cases.

Recommendation: keep the already-chosen route — **Capacitor with a static export** — and additionally publish the Android build as the primary store artifact first (Google review is far more forgiving). Do the "tightening" work in §4, treat the payments question (§5) as a product decision before Apple submission, and follow the phased plan in §8.

Contents

1. Where the app stands today

Area	Status	Detail
Capacitor 8 tooling	DONE	`@capacitor/core|cli|ios|android` installed; config + npm scripts + packaging doc exist. Native `ios/` & `android/` projects **not yet generated** (`npx cap add` never run).
Web manifest	PARTIAL	`public/manifest.webmanifest` exists and is linked in `layout.tsx`, but its two icon entries point to files that don't exist — the manifest is invalid for installability checks.
App icons / splash screens	MISSING	`public/icons/` contains only the logo SVG and a README. No PNG icons, no native icon sets, no splash screens.
Service worker / offline	MISSING	No service worker, no Workbox/Serwist, no offline fallback. Fails PWA installability; weakens the “real app” case in Apple review.
Static-export readiness	BLOCKED	`output: "export"` is prepared but commented out in `next.config.mjs`. Enabling it today breaks: 4 API routes, next-auth, server actions (onboarding + admin), and `next/image` optimization.
Auth (next-auth v4)	BLOCKED	Cookie-session flow assumes same-origin browser context. Inside the Capacitor WebView the origin is `capacitor://localhost` / `https://localhost`, so session cookies to your API domain are third-party and unreliable.
Payments (Stripe)	POLICY RISK	`/api/stripe/checkout` sells digital content. Apple guideline 3.1.1 requires IAP for digital goods purchased in-app (regional carve-outs exist — see §5).
Viewport / safe areas	DONE	`viewportFit: "cover"`, `themeColor`, and `appleWebApp` metadata already configured in `layout.tsx`.
i18n routing	PARTIAL	URL-based `[locale]` routes must provide `generateStaticParams()` everywhere for static export, and locale middleware won't run in the exported bundle.

2. The three routes into the stores

Option A — Pure PWA + TWA (Trusted Web Activity)

Ship the PWA on the web; wrap it for **Google Play only** with Bubblewrap or PWABuilder. The Play app is a thin "Trusted Web Activity" that opens your live site full-screen, verified via a Digital Asset Links file. **There is no equivalent path on iOS** — Apple rejects site-in-a-shell submissions under guideline 4.2 (minimum functionality).

PROS

- Smallest engineering effort; no native projects to maintain
- Keep full Next.js SSR, API routes, next-auth, server actions as-is
- Instant updates — deploy the site, the "app" updates
- Stripe on the web is outside Apple/Google billing rules

CONS

- **No iOS App Store presence** — disqualifying if iOS is a requirement
- Requires online connectivity; needs a solid service worker anyway to pass Play quality bar
- No native APIs beyond web platform (limited push on iOS Safari, no widgets, etc.)

Option B — Capacitor + static export RECOMMENDED · already scaffolded

Build the Next.js app as a static client bundle (`output: "export"` → `out/`), embed it in native iOS/Android shells, and have it call your hosted API over HTTPS. This is what `capacitor.config.ts` (`webDir: "out"`) is already set up for.

PROS

- One codebase → web + iOS + Android
- App works app-like: instant shell start, offline shell possible
- Full native API access via Capacitor plugins (push, secure storage, haptics, IAP)
- Much stronger position in Apple review than a URL wrapper
- UI fixes ship by resubmitting the bundle; no server dependency for the shell

CONS

- Requires decoupling from server features: no API routes, SSR, or server actions in the bundle (§4.2)
- Auth must move to a token flow (§4.3)
- Two build modes to maintain (web SSR vs. native export)
- App updates gated on store review (1–48h typically)
- Apple IAP rules apply to in-app digital purchases (§5)

Option C — Capacitor + remote URL (`server.url`)

Point the native shell at the deployed website. Everything (SSR, API, auth) keeps working because the WebView literally browses your site.

PROS

- Near-zero code changes; fastest path to a running native build
- Full SSR / next-auth / server actions preserved
- Great for internal testing & TestFlight demos

CONS

- **High Apple rejection risk** (4.2 “repackaged website”)
- Useless offline; white screen with no connectivity
- Apple guideline 2.5.2: apps may not materially change behavior via remotely-loaded code — a fully remote app invites scrutiny
- Slower startup (network round-trip before first paint)

Option D — Native rewrite (React Native / Expo)

Best long-term UX and review posture, but a full rewrite of the UI layer. Out of scope for the MVP; noted for completeness. Revisit only if WebView performance of the feed (Three.js, GSAP, Framer Motion) proves inadequate on mid-range Android devices.

Criterion	A · PWA/TWA	B · Cap static	C · Cap remote URL	D · React Native
iOS App Store	✗	✓	△ risky	✓
Google Play	✓	✓	✓	✓
Engineering effort	Low	Medium	Very low	Very high
Offline capability	Medium (SW)	High	None	High
Native APIs (push, storage…)	Web-only	Full	Full	Full
Update speed	Instant	Store review	Instant (risky)	Store review
Apple review risk	n/a	Low–medium	High	Low
Keeps SSR/API routes in-app	✓	✗	✓	n/a

3. Store requirements

Apple App Store

Requirement	Detail
Apple Developer Program	US$99/year (organization enrollment recommended — needs a D-U-N-S number for the company entity behind `ee.taskutark`).
Hardware/tooling	A Mac with current Xcode (16+); signing certificates & provisioning profiles managed in App Store Connect.
Guideline 4.2 — minimum functionality	The app must feel like an app, not a wrapped site. Native touches that materially help: push notifications, offline reading, haptics, native share sheet, secure biometric login.
Guideline 3.1.1 — payments	Digital content purchased in-app must use In-App Purchase (30%/15% commission), with regional exceptions — see §5.
Privacy	Privacy “nutrition label” in App Store Connect, a privacy policy URL, and a `PrivacyInfo.xcprivacy` privacy manifest declaring data collection and required-reason APIs.
Account deletion	If users can create accounts in the app, the app must offer in-app account deletion (guideline 5.1.1(v)).
Sign in with Apple	Required if you offer third-party social login (you have Google login) — plan to add Apple sign-in alongside it.
Assets	1024×1024 icon, iPhone (and iPad if supported) screenshots, description, keywords, support URL, age rating questionnaire.
Review time	Typically 24–48 h; first submissions and education/content apps sometimes get extra scrutiny.

Google Play

Requirement	Detail
Play Console account	US$25 one-time. New **personal** accounts must run a closed test with ≥12 testers for 14 days before production access — an **organization** account avoids this.
Target API level	New apps must target a recent Android API level (API 35 / Android 15 as of the 2025 cycle; Capacitor 8 templates already comply). Ship an **AAB** (app bundle), signed via Play App Signing.
Play Billing	Same principle as Apple: digital in-app content must use Google Play Billing (15% up to $1M/yr revenue).
Data safety form	Declare data collection/sharing (account data, analytics) in the Play Console.
Content & target audience	Content rating questionnaire; if the app targets children/students under 13, Families policy applies — decide the declared age range deliberately.
Assets	512×512 icon, feature graphic (1024×500), ≥2 screenshots, privacy policy URL.
Review time	Usually hours to ~2 days; far fewer “minimum functionality” rejections than Apple.

4. The tightening checklist — engineering work in this repo

4.1 Fix the PWA fundamentals (do first — benefits web too)

- **Icons:** generate real `icon-192.png` / `icon-512.png` (plus a dedicated *maskable* variant with safe-zone padding) from the logo SVG. For the native shells, run `npx @capacitor/assets generate` to produce all iOS/Android icon + splash sets from one 1024px source.
- **Service worker:** add **Serwist** (the maintained successor to `next-pwa`, built for the App Router). Precache the app shell, runtime-cache API responses (stale-while-revalidate for the feed), and add an offline fallback page. This makes the web app installable *and* is your strongest answer to Apple's 4.2 concern.
- **Manifest polish:** keep `start_url: "/feed"` consistent with the locale routing (a redirecting start URL breaks installability audits); add `id`, `orientation`, and screenshots fields. Verify with Lighthouse → "Installable".

4.2 Decouple the client from the server (static-export readiness)

The exported bundle contains **no server**. Everything under `src/app/api/` and every server action must be reachable over HTTPS on your hosted deployment instead:

- Introduce `NEXT_PUBLIC_API_BASE_URL` and route all client fetches through a single helper in `src/lib/api.ts`. On the web it stays relative (`/api/…`); in the native build it points at `https://app.yourdomain/api/…`.
- Convert the onboarding `finishOnboarding` server action (and any other client-triggered actions) into plain API calls when running natively. Keep the seam in one function, per the existing `TODO(team)` convention.
- Exclude `/admin` from the native bundle — it is desktop-first, uses server actions + `requireAdmin()`, and has no business inside the mobile app.
- Add `generateStaticParams()` for every `[locale]` segment so all locales pre-render; locale detection middleware won't run in the export, so default the native app to a stored locale preference.
- Set `images: { unoptimized: true }` for the export build (already documented in `next.config.mjs`).
- **Two build modes:** drive it with an env flag, e.g.

  ```
  // next.config.mjs
  const isNative = process.env.BUILD_TARGET === "native";
  export default {
    output: isNative ? "export" : undefined,
    images: isNative ? { unoptimized: true } : { remotePatterns: [/*…*/] },
  };
  ```

  and change the script to `"cap:sync": "BUILD_TARGET=native next build && cap sync"` so the normal web deployment keeps SSR untouched.

4.3 Auth: move from cookies to tokens for the native shell

- next-auth's cookie session won't work reliably from the `capacitor://localhost` origin (third-party-cookie territory). Expose a token flow on the server (e.g. a JWT issued by your auth endpoint / TT API) and send it as an `Authorization: Bearer` header from the app.
- Store tokens with a secure-storage Capacitor plugin (iOS Keychain / Android Keystore) — **never** `localStorage`, consistent with the project's existing security rule.
- Google OAuth inside a WebView is blocked by Google's policy — use the system browser (`@capacitor/browser` or a Google Sign-In plugin) with a deep-link redirect back into the app.
- Add **Sign in with Apple** (required on iOS once Google login is offered) and an in-app **delete account** action (required by both stores).

4.4 Make it feel native (and pass Apple 4.2)

- **Push notifications** (`@capacitor/push-notifications` + FCM/APNs) — e.g. streak reminders; doubles as review-proof of native functionality.
- **Status bar & splash** (`@capacitor/status-bar`, `@capacitor/splash-screen`) themed to the violet brand; safe-area CSS is already handled via `viewportFit: "cover"` — audit each screen with `env(safe-area-inset-*)` paddings.
- **Deep links:** iOS Universal Links (`apple-app-site-association`) + Android App Links (`assetlinks.json`) so shared studybook URLs open in the app.
- **Haptics** on feed interactions (`@capacitor/haptics`) — cheap, high perceived quality.
- **Offline reading:** cache the user's active studybooks for offline — the single most convincing “this is an app” feature for an education product.
- **Performance audit:** the feed uses Three.js + GSAP + Framer Motion + Lenis; test on a mid-range Android device early. Consider disabling 3D flourishes on low-end devices (`navigator.hardwareConcurrency` / reduced-motion heuristics).

5. Payments: the Stripe problem

> **This is the #1 policy risk in the entire plan.**
> StudyBooks Premium is digital content. Apple guideline 3.1.1 and Google Play's payments policy both require their own billing systems for digital goods bought *inside the app*. Shipping the current Stripe checkout flow in the iOS build is a near-certain rejection.

Your realistic options, in order of pragmatism:

Approach	How it works	Trade-off
"Reader-app" style (recommended for MVP)	Remove all purchase UI from the native builds; users subscribe on the website (Stripe), and the app simply honors the entitlement. Don't link to the checkout from inside the iOS app unless using Apple's external-link entitlement.	Zero store commission; lower conversion (users must find the website); fully policy-safe.
Native IAP / Play Billing	Implement StoreKit + Play Billing (e.g. via RevenueCat, which has a Capacitor SDK) alongside Stripe on the web. Server reconciles entitlements from all three sources.	Best conversion and UX; 15–30% commission; meaningful engineering + entitlement-sync work.
External purchase links / regional carve-outs	The rules have been loosening (US court ruling 2025, EU DMA alternative terms) to allow linking out to web checkout in some regions. Estonia = EU, so DMA terms may apply.	Region-dependent, changes frequently, extra entitlements/compliance. **Re-verify the current rules at submission time** — do not build the plan on this.

Whatever the choice, keep the entitlement check server-side behind one seam (the app asks "is this user premium?", never "which processor did they pay with?").

6. Submission pipeline, step by step

1. **Finish §4.1–4.3** (icons, service worker, static-export readiness, token auth).
2. **Generate native projects:** `npx cap add ios && npx cap add android`, commit them to git.
3. **Assets:** `npx @capacitor/assets generate` from a 1024×1024 source icon + 2732×2732 splash source.
4. **Build & sync:** `npm run cap:sync` (with `BUILD_TARGET=native`), then `npm run cap:ios` / `cap:android`.
5. **Android first:** Android Studio → generate signed **AAB** → Play Console internal testing track → closed testing (mind the 12-tester/14-day rule for personal accounts) → production.
6. **iOS:** Xcode → set team/signing → archive → upload → **TestFlight** (internal, then external testers) → App Store review.

7. **Store listings:** screenshots (device frames per store spec), descriptions, privacy policy URL, data-safety / privacy labels, age rating.
8. **Post-launch cadence:** web deploys stay continuous; native releases batch UI changes on a 1–2 week train. (Capacitor "live update" services exist for JS-only hotfixes but keep them within Apple 3.3.2 limits — no feature-flipping via remote code.)

7. Risks, drawbacks & mitigations

Risk / drawback	Severity	Mitigation
Apple 4.2 rejection ("just a website")	High	Ship push, offline reading, haptics, deep links before first submission; write the review notes explaining native functionality; expect one rejection round in the schedule.
Stripe vs. IAP policy violation	High	§5 — strip in-app purchase UI for MVP (reader-app pattern) or adopt IAP via RevenueCat.
Auth breaks in WebView	High	Token flow + secure storage + system-browser OAuth (§4.3). Test on real devices, not just simulators.
Two build modes drift apart	Medium	Single env flag in `next.config.mjs`; CI job that runs the `BUILD_TARGET=native` build on every PR so export breakage is caught immediately.
Slow store review blocks urgent fixes	Medium	Keep business logic server-side; UI hotfixes via expedited review (Apple grants these sparingly) or a compliant live-update channel.
WebView performance of 3D/animation-heavy feed	Medium	Early testing on mid-range Android; progressive enhancement — static covers instead of Three.js scenes on weak devices.
Maintenance overhead (Xcode/Gradle upgrades, yearly API-level bumps)	Low–medium	Capacitor major upgrades ~yearly; budget a maintenance day per quarter.
Play personal-account testing gate (12 testers / 14 days)	Low	Register the Play account as an **organization** from the start.

8. Recommended phased plan

Phase	Scope	Exit criteria
1 · PWA hardening ~3–5 days	Real icons + maskable variant; Serwist service worker with offline fallback; manifest polish.	Lighthouse “installable” passes; app installs from browser on Android & iOS.
2 · Export readiness ~1 week	`BUILD_TARGET=native` build mode; API base-URL seam; server actions → API calls; `generateStaticParams` for locales; exclude `/admin`.	`BUILD_TARGET=native next build` succeeds; exported app runs fully against the hosted API.
3 · Native shell ~1 week	`cap add ios/android`; assets; splash/status bar; token auth + secure storage; system-browser OAuth; Sign in with Apple; account deletion.	App runs on physical iPhone + Android device with working login and feed.
4 · Store polish ~1 week	Push notifications; deep links; offline reading cache; payments decision from §5 implemented; privacy manifest + data-safety forms; listings.	Internal TestFlight + Play internal track builds approved by the team.
5 · Launch	Play closed → production; TestFlight external → App Store review.	Both store listings live.

Prepared for the StudyBooks (tt-app) codebase, July 2026. Store policies — especially Apple 3.1.1 payment rules and Play target-API requirements — change frequently; re-verify against the official guidelines at submission time.